Maturski rad iz Programiranja

na temu

Primjena AES algoritma na šifriranje tekstualnih dokumenata

Mentor: Učenik: Nina Šmalcelj, prof. Armin Bojanić IV4

Sarajevo, maj 2025. godine

Sadržaj

1. Uvod 3 2. Kriptografija i sigurnost podataka 4 2.1. Osnovni pojmovi Kriptografije 4 2.1.1. Šifriranje i dešifriranje podataka 4 2.1.2. Simetrična i asimetrična kriptografija 5 2.1.3.

Historijski pregled kriptografskih algoritama 6 2.1.4 Sigurnost podataka i potreba za šifriranjem 8 2.2 AES algoritam 9 2.2.1. Historija i razvoj AES algoritma 9 2.2.2 Struktura AES algoritma 9 2.2.3. Proširivanje ključa u AES algoritmu 14 2.2.4. Prednosti i slabosti AES algoritma 17 2.3. Primjena na osiguravanje dokumenata 17 2.3.1. Upotreba u sigurnosti dokumenata 17 2.3.3. Budućnost šifriranja i sigurnost podataka 19 3. ZAKLJUČAK 20 4. BIBLIOGRAFIJA 21 5. BILJEŠKE 22

1. Uvod

Moderno doba sa sobom donosi mnoštvo inovacija koje pomažu današnjoj civilizaciji. Jedan od glavnih preokreta ka olakšanom i jednostavnijem svijetu jeste digitalizacija. Digitalizacija donosi sa sobom mnogo pozitivnih olakšica, od pristupa informacijama do samog razmjenjivanja tih informacija. Međutim, ono što se prije skladištilo i čuvalo fizički, sada se mora osigurati u digitalnoj formi. Kako bi se spriječio neovlašteni pristup tim podacima i osigurala sigurnost tih informacija, uvedeno je korištenje kriptografskih algoritama za šifriranje podataka.

Najpoznatiji i najčešće korišteni algoritam za šifriranje podataka jeste Advanced Encryption Standard (AES), koji se razvio kao zamjena za stariji Data Encryption Standard (DES). AES [1] [2] pruža visoku razinu sigurnosti i efikasnosti, po čemu je stekao titulu globalne norme za zaštitu podataka. U praksi ga susrećemo u zaštiti dokumenata, komunikacijskim protokolima,

financijskih sistemima i mnogim drugim područjima.

Cilj ovog rada jeste objasniti važnost šifriranja u digitalnom svijetu, princip rada AES algoritma i njegovu primjenu u šifriranju tekstualnih dokumenata. U radu će biti obrađeni osnovni pojmovi kriptografije, struktura AES algoritama, te njegova praktična upotreba.

[1] eng. prev. napredna norma šifriranja
[2] eng. prev. norma šifriranja podataka

2. Kriptografija i sigurnost podataka

2.1. Osnovni pojmovi Kriptografije

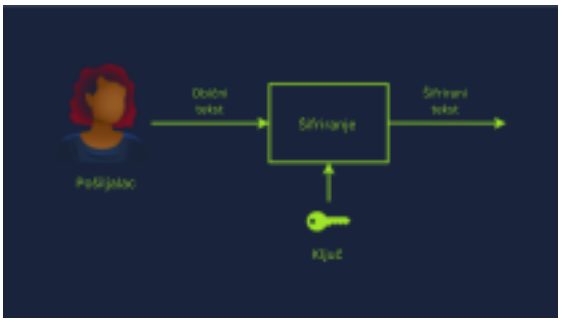
2.1.1. Šifriranje i dešifriranje podataka

Konačni cilj šifriranja jeste da obezbijedi sigurnu komunikaciju u prisustvu protivnika. Šifriranje se može definirati kao praksa i nauka o tehnikama za sigurnu komunikaciju i sigurnost podataka u situacijama kada očekujemo prisustvo nepoželjnih trećih lica. Ova treća lica ne bi trebala imati mogućnost da otkriju ili promijene sadržaj ovih poruka.

Šifriranje je proces u kojem se poruka koju možemo pročitati prebacuje u nečitljivu formu kako bi spriječila neovlaštena lica da je pročitaju. Sje proces u kojem se šifrirana poruka

prebacuje nazad u originalnu (čitljivu) formu. Originalna poruka zove se obični tekst , dok se [3] šifrirana verzija iste te poruke zove šifrirani tekst [4] . Za provođenje procesa šifriranja i dešifriranja potreban nam je ključ.

Ključ u kriptografiji predstavlja niz specifično organiziranih bitova koji je napravljen da šifrira i dešifrira podatke. Svaki ključ je specifičan za dati kôd šifriranja, tako da je svaki pojedinačni ključ unikatan i nikoja dva ključa nisu ista.



Slika 2.1.1. - 1. Prikaz osnovnog principa šifriranje teksta *Slika 2.1.1. - 2. Prikaz osnovnog principa dešifriranje teksta* [5] [6]

Dvije veoma bitne stavke, koje je postavio naučnik Claude Shannon 1945. godine, a koje ocjenjuju koliko je dobar neki algoritam, su konfuzija i difuzija. Konfuzija je mjerilo koliko je komplikovana veza između ključa i dobijenog šifriranog teksta. Dobar i kvalitetan algoritam za šifriranje će povećati utjecaj koji ima promjena jednog dijela ključa na konačni tekst. Želimo da najmanje promjene u ključu rezultiraju što većim promjenama u konačnici.

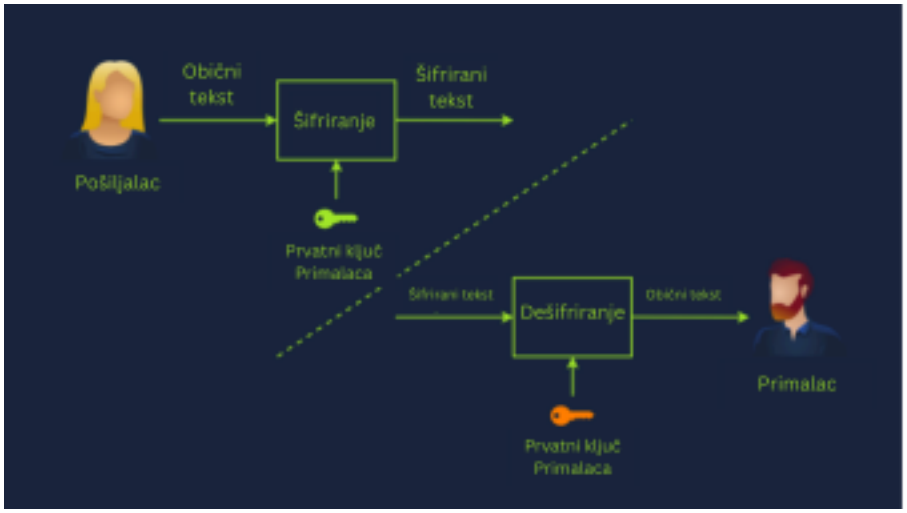
[3] eng. plaintext
[4] eng. ciphertext
[5] Tryhackme i Strategos, “tryhacme”, https://tryhackme.com/room/cryptographybasics, Tryhackme Ltd, 21.10.2024, Web, (01.03.2025. 16:57)
[6] Tryhackme i Strategos, “tryhacme”, https://tryhackme.com/room/cryptographybasics, Tryhackme Ltd, 21.10.2024, Web, (01.03.2025. 16:57)

Difuzija je rasprostranjivanje statističke strukture običnog teksta. Tekst ima statistički obrazac, primjera radi, unutar jedne rečenice sva slova imaju broj koji označava koliko se često pojavljuju unutar teksta, a neke riječi su češće korištene od drugih. Ako se ovi obrasci pojave u šifriranom tekstu, ta bi se ponavljanja mogla analizirati.

Prije prelaska na daljnje oblasti ovog rada, potrebno je razjasniti jednu dilemu u svijetu kriptografije, a to je razlika između šifriranja i heširanja. Pošto smo već objasnili šta je šifriranje, ukratko ćemo objasniti heširanje. Heširanje je proces u kojem dati podatak prebacujemo u nasumičan tekst fiksne dužine koristeći hash funkciju. Glavna razlika između ova dva procesa jeste to što heširanje nema fiksan način za dešifrovanje podatka. Ono je jednosmjerni proces u kojem se osigurava cjelokupnost datog podatka. Iz toga zaključujemo da heširanje nije isto što i šifriranje.

2.1.2. Simetrična i asimetrična kriptografija

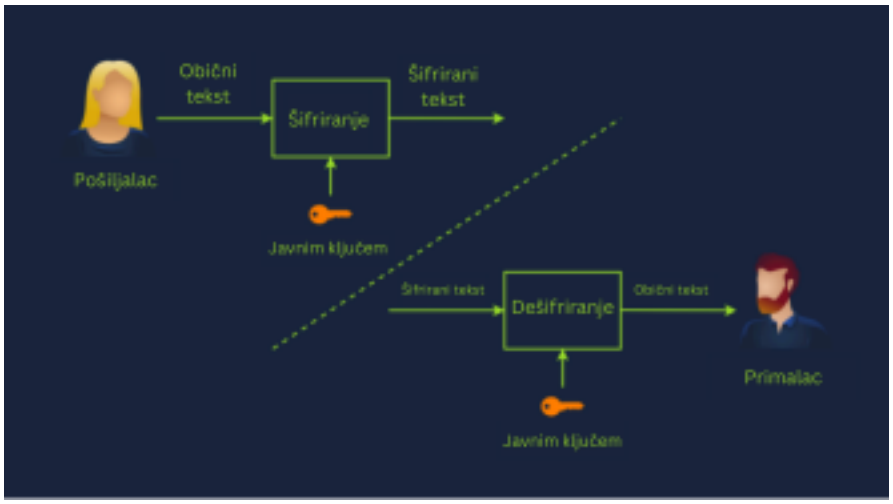
Postoje dvije primarne tehnike šifriranja: simetrično i asimetrično šifriranje. Simetrično šifriranje, ili šifriranje sa privatnim ključem, koristi jedan ključ za šifriranje i dešifriranje podataka. Jedan ključ mijenja dati podatak u šifrirani tekst, i isti taj ključ ga može vratiti u obični tekst. Privatni ključ mora se podijeliti i sa pošiljaocem i s primaocem, što zahtijeva sigurnu metodu za razmjenu ključa.



Slika 2.1.2. - 1. Prikaz simetričnog šifriranja[7]

Asimetrično šifriranje, ili šifriranje sa javnim ključem, koristi matematičku vezu između privatnog i javnog ključa. Javni ključ se dijeli otvoreno i koristi se za šifriranje podataka ili poruka. Podaci koji su šifrirani sa javnim ključem mogu se jedino dešifrirati s odgovarajućim privatnim ključem, koji mora ostati tajan.

[7] Tryhackme i Strategos, "tryhacme", https://tryhackme.com/room/cryptographybasics, Tryhackme Ltd, 21.10.2024, Web, (01.03.2025. 16:57)



Slika 2.1.2. - 2. Prikaz asimetričnog šifriranja[8]

2.1.3. Historijski pregled kriptografskih algoritama

Kriptografija kao tehnika i nauka javila se prije digitalizacije. Već od 1900-ih godina prije nove ere javlja se prva implementacija kriptografije u starom Egipatskom carstvu, gdje su pronađeni nestandardni hijeroglifi uklesani u zidove grobnica. Nastavljajući s razvojem ove tehnike, u 1500-im godinama prije nove ere pronađeni su zapisi u glini u Mezopotamiji koji su čuvali šifrirana pisma. Već od 650-ih godina prije nove ere pronađeni su prvi tragovi šifriranog i običnog teksta, gdje su Spartanci koristili kriptografiju da šifruju pisma za vojnu komunikaciju. Proces šifriranja je započinjao zapisivanjem poruke na komadu kože koji je bio zamotan oko heksagonalnog drvenog štapa kojeg su zvali skital. Kada se koža obmota oko skitala određene veličine, slova bi se poredala i formirala pravilnu poruku, međutim, kada se poruka skine sa skitala, ona bi se vratila u početno stanje, to jest, šifrirani tekst. Zapravo, skital određene veličine koja odgovara za dešifriranje poruke može se smatrati prvim privatnim ključem. Naravno, mora se spomenuti i Cezarovo šifriranje, koje je nastalo između 100. i 44. godine p.n.e., a danas se koristi kao najosnovniji oblik šifriranja za objašnjavanje osnovnih koncepata u kriptografiji.

Slika 2.1.3. - 1. Slika skitala, metoda šifriranja za spartanac[9]

[8] Tryhackme i Strategos, "tryhacme", https://tryhackme.com/room/cryptographybasics, Tryhackme Ltd, 21.10.2024, Web, (01.03.2025. 16:57)
[9] n.a., "cachesleuth", https://www.cachesleuth.com/scytale.html, n.p., n.d., Web, (04.03.2025 20:17)

Iz srednjeg vijeka datiraju i prvi tragovi probijanja šifriranog teksta. 800. godine, arapski matematičar Al-Kindi izumio je tehniku analize frekvencija za razbijanje šifri, što predstavlja jedan od najsjajnijih proboja u kriptoanalizi. Analiza frekvencija koristi jezičke podatke, kao što su učestalost pojedinih slova ili uparivanje slova, dijelove govora i konstrukciju rečenica, da rekonstruira privatni ključ za dešifriranje. Tehnike analize frekvencija mogu se koristiti za ubrzavanje napada grubom silom (eng. *brute-force attacks*), u kojima se dešifratori sistemski pokušavaju dešifrirati kodirane poruke primjenjujući potencijalne ključeve u nadi da će se eventualno pronaći tačan ključ. Al-Kindiova djela također govore o tehnikama kriptoanalize za polialfabetske šifre koje mijenjaju obični tekst šifriranim koristeći više alfabeta za dodatni sloj sigurnosti. 1467. godine javlja se Leon Battista Alberti, koji se smatra ocem moderne kriptografije. On je u svojim djelima razmatrao i analizirao šifriranje koje koristi više alfabeta kao najjaču formu šifriranja u srednjem vijeku.

Slika 2.1.3. - 2. Slika Arapskog matematicara Al-Kindi[10]

Počevši od 20. stoljeća, dolazi do ubrzanog razvoja šifriranja i kriptografije. Već od početka Prvog svjetskog rata šifriranje dobija značajnu ulogu u vojsci. Kako je Njemačka napredovala sa šifriranjem svojih poruka unutar vojske, tako je Engleska razvijala i poboljšala svoje sposobnosti u probijanju njihovih šifri. 1917. godine amerikanac Edward Hebern je napravio prvu kriptografsku rotor mašinu koja kombinira strujna kola i mehaničke dijelove pisaće mašine da automatizuje šifriranje poruka. Korisnik bi mogao napisati svoju poruku u običnom tekstu, a Edwardova mašina bi automatski šifrirala poruku u šifrirani tekst. Šifrirani tekst bi se kasnije mogao dešifrirati tako što se ručno okrene kolo rotora, i tada upisivanjem šifriranog teksta, mašina bi automatski izbacivala poruku u običnom dešifrovanom tekstu.

[10] The Editorial team, "Musilm Heratige", https://muslimheritage.com/al-kindi/, n.p., 06.05.2007., Web, (04.03.2025. 20:23)

Slika 2.1.3. - 3. Edward Herernova rotor mašina Slika 2.1.3. - 4. Enigma Masina [11] [12]

Već 1918. godine, nijemac Arthur Scherbius konstruira Enigma mašinu kao poboljšanog nasljednika Hebernove mašine. Također, Enigma je koristila rotor kola za šifriranje i dešifriranje. Međutim, slično kao Hebernov izum, Enigma je zahtijevala da se podijele

postavke za kalibraciju mašine i privatni ključevi koji su bili podložni špijunaži. Srž njemačke vojske i njihove komunikacije bazirala se oko Enigma mašine tokom Drugog svjetskog rata, sve dok Alan Turing zajedno s saveznicima nije uspio probiti kroz njemački sistem. 1975. godine, IBM razvija Data Encryption Standard (DES), algoritam za šifriranje koji će postati prvi certificirani sistem za šifriranje koji je uspostavio National Institute of Standards and [13] Technology. I sve do 2001. godine razvijaju se mnogi načini šifriranja, kao što su RSA i Diffie-Hellman algoritmi. 2001. godine, Advanced Encryption Standard (AES) dolazi na snagu i uspostavlja DES algoritam kao novi certificirani standard šifriranja.

Polje kriptografije i šifriranja stalno se mijenja, a budućnost pruža mnogo prostora za naučnike da razvijaju nove algoritme i standarde, jer se kvantni računari sve brže razvijaju. Ovo ne predstavlja problem sve dok kvantni računari ne bude dovoljno razvijen da može probiti današnji algoritmi šifriranja u roku od nekoliko sekundi.

2.1.4 Sigurnost podataka i potreba za šifriranjem

U današnjem vremenu sigurnost podataka je postala ključna tema. Sve više podataka cirkulira digitalnim svijetom i sve je veća potreba za novim i naprednijim načinima da se ti podaci osiguraju. Šifriranje pomaže da se naši podaci osiguraju i ostanu nepromijenjeni, a šifriranje je od velikog značaja ne samo za lične podatke, već i za poslovne informacije, financijske transakcije i državne tajne.

[11] Ralph Simpson, ”Computer History”, https://computerhistory.org/blog/before-enigma-breaking-the-hebern-rotor-machine/, 08.08.2017., Web, (04.03.2025 20:09)
[12] John Jackson, “tnmoc”, https://www.tnmoc.org/bh-2-the-enigma-machine, n.p., n.d., Web, (04.03.2025 20:38)
[13] National Institute of Standards and Technology - Nacionalni institut za standarde i tehnologiju je agencija Ministarstva trgovine Sjedinjenih Država čija je misija promovirati američke inovacije i industrijsku konkurentnost.

2.2 AES algoritam

2.2.1. Historija i razvoj AES algoritma

Advanced Encryption Standard (AES) je nastao 1998. godine, napravljen je da zamijeni tadašnji standardni algoritam za šifriranje, Data Encryption Standard (DES). AES su napravili dva belgijanca, Vincent Rijmen i Joan Daemen. National Institute of Standards and Technology (NIST) je organizirao natječaj da nađu novi i sigurniji standard za šifriranje. Tako je 2001. godine AES algoritam pobijedio i proglašen je kao novi standard za šifriranje. AES je nadogradio nekoliko aspekata koji su bili glavni problemi u DES algoritmu, poput veće veličine ključeva i blokova, što ga je ujedno činilo dosta sigurnijim. AES algoritam se danas koristi u šifriranju internet komunikacije, čuvanju osjetljivih podataka i šifriranju podataka. Sve velike firme, od Googlea do Mete, koriste ovaj algoritam kako bi čuvale svoje podatke i

informacije na sigurnom.

2.2.2 Struktura AES algoritma

AES je takozvana blok-šifra (*eng. block cypher*), što znači da uzima blok običnog teksta sa konstantnom veličinom bitova i dobija blok šifriranog teksta sa istom veličinom bitova. AES algoritam koristi blokove od 128 bita i ključeve dužine od 128, 192 ili 256 bita, što mu omogućava da ima različite nivoe sigurnosti u zavisnosti od dužine ključa. Struktura AES algoritma bazira se na nekoliko osnovnih operacija koje se primjenjuju na podatke nakon fiksnog broja krugova (eng. *rounds*), zavisno od dužine ključa.

Veličina ključa	Broj rundi
128	10
192	12
256	14

Tabela 2.2.2. - *1. Broj rundi za šifriranje na osnovu veličine ključa*

Ova struktura omogućava visok nivo sigurnosti, jer u svakom krugu dolazi do složenih promjena u podacima, čineći ih praktično neprepoznatljivim bez ispravnog ključa za dešifriranje.

SubBytes je operacija koja mijenja svaki bajt u blokovima podataka prema unaprijed definisanoj tabeli (S-box), što predstavlja zamjenu s ciljem povećanja složenosti šifriranja.

ShiftRows je operacija pomjera redove podataka, čime se postavlja osnov za složenije transformacije tokom sljedećih koraka.

MixColumns operacija transformiše svaku kolonu bloka koristeći matematičke operacije koje omogućavaju šifriranje na nivou kolona, što dodatno komplikuje dešifriranje. AddRoundKey je faza u kojoj se odgovarajući ključ za odgovarajući krug dodaje na podatke koristeći XOR operaciju, čime se povezuje šifriranje teksta s ključem.

AES je karakterističan po tome da je efikasan, brz i otporan na napade poput diferencijalne i linearne kriptoanalize.

Šifriranje pomoću AES algoritma počinje sa podatkom od 16 bajta koje želimo šifrirati. AES

ove bajte podijeli u mrežu od 4 kolone i 4 reda, a ova mreža se zove stanje algoritma i prevodi sve podatke unutar mreže u heksadecimalni brojni sistem. AES će svo svoje šifriranje vršiti na ovoj mreži.

b_0	b_4	b_8	b_{12}
b_1	b_5	b_9	b_{13}
b_2	b_6	b_{10}	b_{14}
b_3	b_7	b_{11}	b_{15}

Tabela 2.2.2. - 2. Prikaz stanja algoritma

Prije početka šifriranja, vrši se proširivanje originalnog ključa, iz koje ćemo dobiti dodatne ključeve za svaku rundu šifriranja. Primjera radi, ako koristimo ključ od 128 bita, generisat ćemo dodatnih deset ključeva, pa ćemo imati ukupno jedanaest ključeva koje koristimo tokom šifriranja.

AddRoundKey

Na početku se vrši dodavanje prvog ključa. Ključevi se također razdvoje na mreže od 4 kolone i 4 reda, te se vrši AddRoundKey. Na bajte iz običnog teksta dodaje se heksadecimalni broj iz ključa K0, koji ima istu poziciju kao bajt na koji se dodaje.

04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

+ =

a0	88	23	2a
fa	54	a3	6c
fe	2c	39	76
17	b1	39	05

a4	68	6b	02
9c	9f	5b	6a
7f	35	ea	50
f2	2b	43	49

Tabela 2.2.2. - 3. Blok običnog teksta Tabela 2.2.2. - 4. Blok K_0 Tabela 2.2.2. - 5. Blok novog ključa stanja

Nakon završetka ovog dijela postupka, završen je prvi i jedini predkorak potreban da se pređe na glavne runde. Sada počinje odbrojavanje rundi šifriranja, zavisno od toga koju veličinu ključa smo odabrali.

```
Ulaz:
   state[4][4] // Trenutna 4x4 matrica bajtova (State)
   roundKey[4][4] // Ključ za trenutnu rundu (Round Key), takođe 4x4 matrica bajtova
Za svaki red i od 0 do 3:
   Za svaki stupac j od 0 do 3:
      state[i][j] = state[i][j] XOR roundKey[i][j]
Izlaz:
   Ažurirana matrica state nakon što je primijenjena operacija AddRoundKey
```

Svaka runda osim zadnje ima četiri glavna koraka koja uvijek idu istim redom:

1. SubBytes
2. ShiftRows
3. MixColumns
4. AddRoundKey

U **S**ubBytes koraku, svaki bajt se mijenja drugim pomoću S-boxa. S-box je predefinisana tabela u kojoj imamo x redova i y kolona. Uzimajući bajt po bajt iz stanja algoritma, uzimamo heksadecimalnu vrijednost jednog bajta, razdvajamo ga na dva dijela i prvu vrijednost poredimo sa odgovarajućim x redom, a zatim drugu sa y kolonom. Na mjestu susretanja reda i kolone uzimamo vrijednost koja se nalazi u S-Box tabeli.

```
Ulaz:
   state[4][4] // 4x4 matrica bajtova (State)
   SBox[256] // Zamjenska kutija (Substitution Box) – niz od 256 elemenata Za
svaki red i od 0 do 3:
   Za svaki stupac j od 0 do 3:
      byte = state[i][j] // Uzmi trenutni bajt iz state
      state[i][j] = SBox[byte] // Zamijeni bajt koristeći SBox
Izlaz:
   Ažurirana matrica state nakon zamjene svih bajtova pomoću SBox-a
```

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Slika 2.2.2. - 1. S-box tabela[14]

a4	68	6b	02
9c	9f	5b	6a
7f	35	ea	50
f2	2b	43	49

=

49	45	7f	77
de	db	39	02
d2	96	87	53
a1	f1	1a	3b

Tabela 2.2.2. - 6. Stanje algoritma Tabela 2.2.2. - 7. Blok nakon SubBytes

ShiftRows je najjednostavniji korak u algoritmu. Potrebno je svaki red pomjeriti u lijevo za predefinisani broj puta: prvi red se ne pomjera, drugi red se pomjera jednom u lijevo, treći red se pomjera dva puta u lijevo, a četvrti red se pomjera tri puta u lijevo.

```
Ulaz:
   state[4][4] // 4x4 matrica bajtova
Za svaki red i od 1 do 3:
   PrivremeniRed[4]
   Za svaki stupac j od 0 do 3:
      k = (j + i) % 4
      PrivremeniRed[j] = state[i][k]
   // Kopiraj nazad u state
   Za svaki stupac j od 0 do 3:
      state[i][j] = PrivremeniRed[j]
Izlaz:
Ažurirana matrica state nakon rotacije redova
```

[14] Athanasios Kakarountas, “Research gate”, https://www.researchgate.net/figure/S-box-substitution-values-for-the-byte-xy-in-hexadecimal-format_fig4_220091765, n.p., n.d., Web, (14.03.2025 18:52)

=

49	45	7f	77
de	db	39	02
d2	96	87	53
a1	f1	1a	3b

49	45	7f	77
db	39	02	de
87	53	d2	96
3b	a1	f1	1a

Tabela 2.2.2. - 8. Stanje algoritma Tabela 2.2.2. - 9. Stanje algoritma nakon ShiftRows

Sljedeći korak je MixColumns. U ovom koraku svaka kolona se transformiše pomoću specifične matrice. Ova matrica se koristi za pomeranje i miješanje podataka unutar kolona, čime se postiže veća sigurnost kroz difuziju. Svaka kolona se tretira kao polinom u Galoovom polju GF(2^8).[15]

49	45	7f	77
db	39	02	de
87	53	d2	96
3b	a1	f1	1a

Tabela 2.2.2. - 10. Stanje algoritma

* =

02	03	01	01
01	02	03	01
01	01	02	03
03	01	01	02

matrica za MixColumns

58	33	db	1b
4d	63	e7	6b
ca	22	ca	b0
f1	fc	a8	e5

Tabela 2.2.2. - 12. Novo stanje algoritma

Tabela 2.2.2. - 11. AES-128

Ulaz:

```
    state[4][4] // 4x4 matrica bajtova

Funkcija xtime(byte):
    // Pomnoži bajt sa 2 u GF(2^8)
    Ako (byte & 0x80) == 0x80:
```

```
        vrati ((byte << 1) ^ 0x1B) & 0xFF
    Inače:
        vrati (byte << 1) & 0xFF
```

[15] **Galoovo polje** (ili *GF polje*, od engleskog *Galois Field*) je matematička struktura koja se koristi u mnogim kriptografskim algoritmima, uključujući AES, za obavljanje aritmetičkih operacija. Ime je dobilo po francuskom matematičaru *Évaristeu Galoisu*, koji je istraživao algebarske strukture i postavio temelje za teoriju polja. Polje je skup brojeva na kojem su definirane dvije osnovne operacije: sabiranje i množenje, koje zadovoljavaju određena pravila (komutativnost, asocijativnost, postojanje neutralnih elemenata, i obrnuti elementi). GF(p) označava Galoovo polje sa p elementa, gdje je p prost broj. Primjera radi, GF(2) je polje sa dva elementa: 0 i 1, i koristi se u binarnoj aritmetici. Sabiranje (i oduzimanje) je isto kao XOR operacija između bitova. Množenje je složenija operacija, koja se često izvodi uz pomoć polinoma modula (npr. modifikovanog nereducibilnog polinoma).


```
Funkcija Multiply(a, b):
    // Pomnoži bajtove a * b u GF(2^8)
    rezultat = 0
    Za i od 0 do 7:
        Ako (b & 1) == 1:
            rezultat ^= a
        visak = a & 0x80
        a = (a << 1) & 0xFF
        Ako visak == 0x80:
            a ^= 0x1B
        b >>= 1
    vrati rezultat

Za svaki stupac j od 0 do 3:
    a0 = state[0][j]
    a1 = state[1][j]
    a2 = state[2][j]
    a3 = state[3][j]

    state[0][j] = Multiply(a0, 2) ^ Multiply(a1, 3) ^ a2 ^ a3
    state[1][j] = a0 ^ Multiply(a1, 2) ^ Multiply(a2, 3) ^ a3
    state[2][j] = a0 ^ a1 ^ Multiply(a2, 2) ^ Multiply(a3, 3)
    state[3][j] = Multiply(a0, 3) ^ a1 ^ a2 ^ Multiply(a3, 2)

Izlaz:
    Ažurirana matrica state nakon primjene MixColumns transformacije
```

Poslije MixColumns operacije, radi se AddRoundKey koji smo objasnili ranije, i prelazi se na narednu rundu. AES algoritam ponavlja iste operacije sve do zadnje runde, kada se samo izbaci faza MixColumns. Tada se proces šifriranja pomoću AES algoritma završava.

Za dešifriranje podataka AES algoritmom prolazi se kroz iste korake samo u obrnutom redoslijedu. Jedini izuzeci su u koracima MixColumns i SubByte, gdje se koriste njihove suprotne opcije: Inverse MixColumns i Inverse SubByte. Proces dešifriranja je sljedeći: 1. AddRoundKey

2. Inverse MixColumns

3. ShiftRows
4. Inverse SubByte

Inverse MixColumns je sličan kao njegov odgovarajući korak u šifriranju. Jedina razlika između ova dva procesa jeste u matrici koja se koristi. Da budemo precizni, koristi se inverzna matrica originalne.


```
Ulaz:
   state[4][4] // 4x4 matrica bajtova

Funkcija Multiply(a, b):
   // Pomnoži bajtove a * b u GF(2^8)
   rezultat = 0
   Za i od 0 do 7:
      Ako (b & 1) == 1:
         rezultat ^= a
      visak = a & 0x80
      a = (a << 1) & 0xFF
      Ako visak == 0x80:
         a ^= 0x1B
      b >>= 1
   vrati rezultat

Za svaki stupac j od 0 do 3:
   a0 = state[0][j]
   a1 = state[1][j]
   a2 = state[2][j]
   a3 = state[3][j]

      state[0][j] = Multiply(a0, 0x0E) ^ Multiply(a1, 0x0B) ^ Multiply(a2, 0x0D) ^ Multiply(a3,
0x09)
      state[1][j] = Multiply(a0, 0x09) ^ Multiply(a1, 0x0E) ^ Multiply(a2, 0x0B) ^ Multiply(a3,
0x0D)
      state[2][j] = Multiply(a0, 0x0D) ^ Multiply(a1, 0x09) ^ Multiply(a2, 0x0E) ^ Multiply(a3,
0x0B)
      state[3][j] = Multiply(a0, 0x0B) ^ Multiply(a1, 0x0D) ^ Multiply(a2, 0x09) ^ Multiply(a3,
0x0E)

Izlaz:
   Ažurirana matrica state nakon primjene InvMixColumns transformacije
```

58	33	db	1b

4d	63	e7	6b
ca	22	ca	b0

f1	fc	a8	e5

0e	0b	0d	09
09	0e	0b	0d
0d	09	0e	0b
0b	0d	09	0e

Tabela 2.2.2. - 14. AES-128 matrica za Inverse MixColumns

49	45	7f	77
db	39	02	de
87	53	d2	96
3b	a1	f1	1a

Tabela 2.2.2. - 15. Novo stanje algoritma

* =

Tabela 2.2.2. - 13. Stanje algoritma

Proces Inverse SubByte je potpuno isti kao njegov odgovarajući korak u šifriranju, pri čemu se umjesto obične S-box tabele koristi inverzna S-box tabela.

x \ y	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
4	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
9	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

Slika 2.2.2. - 2. Inverse S-box[16]

```
Ulaz:
   state[4][4] // 4x4 matrica bajtova
   InvSBox[256] // Inverzna zamjenska kutija (Substitution Box) – niz od 256 elemenata

Za svaki red i od 0 do 3:
   Za svaki stupac j od 0 do 3:
      byte = state[i][j] // Uzmi trenutni bajt iz state
      state[i][j] = InvSBox[byte] // Zamijeni bajt koristeći InvSBox

Izlaz:
   Ažurirana matrica state nakon zamjene svih bajtova pomoću InvSBox-a
```

2.2.3. Proširivanje ključa u AES algoritmu

Veoma je bitno spomenuti da se prilikom šifriranja podataka AES algoritmom prvo vrši proces koji se zove proširivanje ključa. U ovom procesu se iz prvobitnog ključa stvaraju i nastaju dodatni ključevi. Broj dodatnih ključeva zavisi od veličine prvobitnog ključa, da li je 128, 192, ili 256-bitni ključ, iz razloga što veličina ključa određuje kroz koliko rundi

algoritam prolazi prilikom šifriranja. U slučaju AES-128 algoritma vrši se proširivanje ključa kako bi dobili dodatnih deset ključeva, pa se ukupan broj ključeva širi na jedanaest.

Prilikom proširivanje ključa možemo posmatrati svaku kolonu ključa kao 32-bitnu riječ. Nama je potrebno da generišemo četrdeset četiri, 32-bitne riječi kako bismo imali dovoljno ključeva za AES-128 algoritam.

[16] Athanasios Kakarountas, "Research gate", https://www.researchgate.net/figure/nverse-S-box-substitution-values-for-the-byte-xy-in-hexadecimal-format_fig5_220091765, n.p., n.d., Web, (14.03.2025 18:52)

15

2b	28	ab	09
7e	ae	f7	cf
15	d2	15	4f
16	a6	88	3c

W_0 W_1 W_2 W_3

Tabela 2.2.3. - 1. Prikaz prvobitnog ključa sa imenovanim kolonama

Proširivanje ključa vrši se u tri koraka:

1. RotWord
2. SubWord
3. Rcon

Formula po kojoj zavisi koje se kolone uzimaju je sljedeća. Neka nam je W_i trenutna prazna kolona koju posmatramo i koju treba da popunimo. Prvo se vrši RotWord, tu uzimamo kolonu W_{i-1} nad kojom se takođe vrši SubWord, a u fazi Rcon prvo se sabiraju kolone W_{i-4}i nova modifikovana W_{i-1}. Nad tim rezultatom vrši se sabiranje sa odgovarajućom kolonom iz predefinisane Rcon matrice.

RotWord

Primjera radi, uzmimo AES-128 algoritam i izvršićemo proširenje ključa na njegov primarni ključ, koji je dat u tabeli 2.2.3.-1. W_i u ovom primjeru na početku će biti W_4 koja je naredna potpuno prazna kolona. Uzimajući W_{i-1} vidimo da je to kolona W_3. Nad kolonom W_3 se vrši RotWord, što samo pomjera prvi član kolone na kraj i pomjera ostale članove za jedno mjesto više.

09
cf
4f

3c

=

cf
4f
3c

09

Tabela 2.2.3. - 2. RotWords na kolonu W3

Ulaz:


```
   word[4] // 4-bajtni niz (32-bitni word) koji treba rotirati

Za i od 0 do 3:
   word[i] = word[(i + 1) % 4] // Rotiraj elemente ulijevo

Izlaz:
   Ažurirani word nakon rotacije
```

SubWord

Korak **SubWord** je potpuno isti kao proces **SubByte** tokom šifriranja. Jedina razlika je u tome što se **SubWord** vrši samo na jednoj koloni. Koristi istu **S-box** tabelu i isti princip da se kolona promijeni.

			01
cf	=		
4f		8a	
3c		84	
09		eb	

Tabela 2.2.3. - 3. Primjena koraka SubWord

```
Ulaz:
   word[4] // 4-bajtni niz (32-bitni word) koji treba podvrgnuti SubBytes operaciji

Za i od 0 do 3:
   word[i] = SBox[word[i]] // Zamijeni svaki bajt koristeći SBox

Izlaz:
   Ažurirani word nakon primjene SubBytes operacije na svaki bajt
```

Rcon

Ovo je posljednji korak u procesu proširivanje ključa. Rcon koristi formulu $W_{i-4} + W_{i-1}$ da bi izračunao finalnu vrijednost kolone na kojoj radi. Međutim, Rcon je specifičan u slučaju kada se ova faza izvršava za prvu kolonu novog ključa. U tom slučaju, na prethodni zbir ($W_{i-4} + W_{i-1}$) dodaje se kolona iz prava Rcon tabele. Rcon tabela ima 10 kolona, i prilikom korištenja jedne od kolona iz ove tabele, ta se kolona više ne koristi i prelazi se na sljedeću pri narednoj upotrebi.

01	02	04	08	10	20	40	80	1b	36
00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00

Tabela 2.2.3. - 4. Konstantna acon tabela

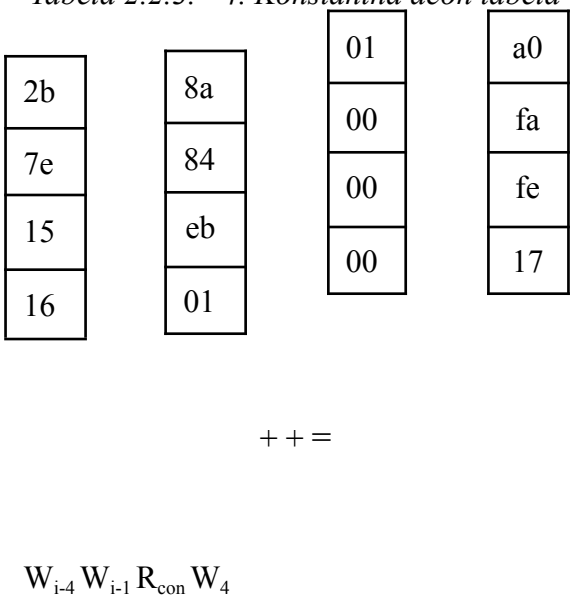


Tabela 2.2.3. - 5. Posljedni korak Rcon na prvu kolonu novog ključa

```
Funkcija Rcon(runda):
   Ako runda == 0:
      vrati 0x00000000
   Inače:
      rcon = 0x01
      Za i od 1 do runda-1:
         rcon = xtime(rcon)
      vrati rcon

Funkcija xtime(byte):
   // Pomnoži bajt sa 2 u GF(2^8)
   Ako (byte & 0x80) == 0x80:
      vrati ((byte << 1) ^ 0x1B) & 0xFF
```

```
Inače:
  vrati (byte << 1) & 0xFF
```

Poslije koraka Rcon, proces proširivanja ključa se ponavlja dok se ne napravi svih četrdeset četiri, 32-bitnih kolona. Nakon što je poslednja kolona kreirana, proces proširivanja ključa se završava i prelazi se na proces šifriranja.

2.2.4. Prednosti i slabosti AES algoritma

Najveća prednosti AES algoritma je njegova sigurnost. Kao što je prethodno objašnjeno, AES koristi složen sistem za proširivanje ključa i višestruke transformacije prilikom procesa šifriranja, što ga čini otpornim na većinu poznatih napada. Takođe, efikasnost algoritma je

znatno bolja od prethodnih algoritama poput DES-a, jer koristi paralelnu obradu i radi na bajtovima umjesto na bitovima. Fiksna veličina bloka omogućava standardizaciju u šifriranju i dešifriranju bez potrebe za prilagođavanjem dužine podataka i dužine ključeva, te pruža korisniku različite stepene sigurnosti, gdje čak i najosnovniji ključ od 128 bita ne može biti probijen.

Iako AES algoritam nije kriptografski slomljen, ima svojih mana. Na primjer, može biti ugrožen putem napada analizom potrošnje energije, elektromagnetnog zračenja ili vremenskih analiza, kao što je napad korištenjem keš memorije. Takođe, AES je prilično nepraktičan u uređaje s ograničenim resursima, poput IoT uređaja ili pametnih telefona, zbog svoje složenosti i zahtjeva za resursima. Implementacija samog algoritma može biti složena i podložna greškama, što dodatno otežava njegovu primjenu u određenim situacijama.

2.3. Primjena na osiguravanje dokumenata

2.3.1. Upotreba u sigurnosti dokumenata

Upotreba AES algoritma u vlastite svrhe je veoma jednostavna. Počevši od činjenice da se sami algoritam može napisati u bilo kojem jeziku koji mi želimo. Ja sam odabrao Python jer on posjeduje ugrađenu biblioteku “pycryptodome” u kojoj imamo sve potrebne funkcije da izvršimo šifriranje vlastitog dokumenta. Prvi korak jeste uključivanje biblioteke unutar našeg projekta

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes
```

Uključivanjem ovih biblioteka dobijamo mogućnost pozivanja šifriranja nad našim tekstom. Funkcije “pad” i “unpad” omogućavaju nam da proširimo ili izbrišemo nepotrebne podatke iz bloka koji se šifrira. Pošto je AES algoritam podijeljen u tačno određene veličine blokova, ove

dvije funkcije nam pomažu da dostignemo te veličine ili da ih uklonimo prilikom dešifriranja.

```
key = get_random_bytes(16)
cipher = AES.new(key, AES.MODE_CBC)
```

Korištenjem ugrađenih funkcija, kao što je get_random_bytes, omogućava nam da dobijemo nasumičnu vrijednost ključa za šifriranje, koji je tačno 16 bajta. Pored toga, bitno je naglasiti način rada šifriranja koji će AES algoritam koristiti. Ovdje se koristi samo radi formalnosti i bira se Cipher Block Chaining (CBC) način rada. [17]

```
data = "Ovo je nasa poruka za sifriranje"
encrypted_data = cipher.encrypt(pad(data, AES.block_size))
```

[17] eng. prev. šifriranje ulančavanjem blokova

Unutar varijable možemo spremiti naš tekst koji će biti šifriran, a pozivom funkcije cipher.encrypt zapravo vršimo šifriranje poruke. Kao što smo objasnili, funkcija pad će proširiti naš blok teksta na odgovarajuću veličinu u slučaju da je manji od 128 bita.

```
iv = cipher.iv
cipher_decrypt = AES.new(key, AES.MODE_CVC, iv)
decrypted_data = unpad(cipher_decrypt.decrypt(encrypted_data), AES.block_size)
```

Prilikom dešifriranja potrebno je sačuvati prvih 16 bajta šifriranog podatka u inicijalizacijski vektor (IV), gdje ponovo definišemo način rada AES algoritma. Zatim dešifriramo podatak koristeći unpad, koji će ukloniti dodatne bajte sa kraja, ostavljajući samo originalni oblik podataka.

2.3.2. Upotreba u industriji

AES je postao industrijski standard za šifriranje podataka i koristi se širom svijeta u različitim industrijama zbog svoje efikasnosti, sigurnosti i brzine. Nekoliko ključnih industrija u kojima se koristi uključuju: finansijsku, zdravstvenu, vladinu i vojnu industriju, e-trgovinu, internet, digitalna prava i Cloud Computing.[18]

U finansijama AES se koristi za zaštitu osjetljivih podataka, kao što su kreditne kartice, i primjenjuje se u šifriranju transakcija u decentraliziranim mrežama, to jest u transakcijama koje se obavljaju u kriptovalutama.

Primarna upotreba u zdravstvu je u zaštiti podataka o pacijentima u skladu s regulativama poput Health Insurance Portability and Accountability Act (HIPAA) u SAD-u. Koristi se za [19] šifriranje elektronskih medicinskih kartona koji se razmjenjuju između bolnica, klinika i zdravstvenih ustanova.

Vladine organizacije i vojska koriste AES za šifriranje informacija tokom komunikacije. U

e-trgovinama i internetu AES se koristi u SSL/TLS protokolima za šifriranje podataka između web servera i klijenata tokom online kupovine, zaštite ličnih podataka, brojeva kreditnih kartica i drugih povjerljivih podataka.

Zaštita autorskih prava (DRM) koristi AES za šifriranje i zaštitu filmova, muzike, softverskih aplikacija i drugih digitalnih medija od nelegalnog kopiranja i distribucije.

[18] eng. prev. računarstvo u oblaku

[19] eng. prev. zakon o prenosivosti i odgovornosti zdravstvenog osiguranja iz 1996. godine je akt Kongresa Sjedinjenih Država koji je donio 104. Kongres Sjedinjenih Država, a potpisao ga je predsjednik Bill Clinton 21. avgusta 1996. godine.

U računarstvu u oblaku AES osigurava podatke koji se čuvaju u oblačnim sistemima, kao podatke na serveru, tako i podatke u transportu.

2.3.3. Budućnost šifriranja i sigurnost podataka

Budućnost šifriranja i sigurnosti podataka je neizvjesna, ali s obzirom na tehnologiju koja brzo evoluira, možemo očekivati nekoliko ključnih trendova koji će oblikovati način na koji štitimo podatke u narednim godinama. Tehnološki napredak, posebno u oblastima kao što su kvantni računari i umjetna inteligencija, pridonose razvoju ove oblasti u pozitivnom smjeru.

Kvantni računari su najveći izazov koji čeka šifriranje podataka u budućnosti. Oni mogu rješavati probleme koji su izuzetno teški za klasične računare s izuzetnim brzinama. Ova revolucionarna tehnologija prijeti postojećim algoritmima za šifriranje, ali također će imati veliki doprinos u razvoju novih i kompleksnijih algoritama.

Uloga umjetne inteligencije postat će ključna u oblasti zaštite podataka i sigurnosti. Ona će poboljšati sposobnosti detekcije napada u realnom vremenu i unaprijediti postojeće algoritme za šifriranje, te razvoj novijih i sigurnijih metoda.

3. ZAKLJUČAK

AES algoritam je postao ključni alat za šifriranje podataka zbog svoje efikasnosti, sigurnosti i otpornosti na napade. Korištenjem 128, 192 i 256-bitne ključeva, pruža visok nivo zaštite za tekstualne dokumente i druge informacije, omogućavajući sigurno čuvanje i prijenos podataka. Iako AES pruža odličnu zaštitu, izazovi poput sigurnosti ključeva i budući razvoj kvantnih računara ukazuju na potrebu za daljnjim unapređenjem i istraživanjem otpornijih algoritama. Pored tih izazova, AES ostaje jedan od najsigurnijih i najšire korištenih algoritama u oblasti digitalne sigurnosti.

4. BIBLIOGRAFIJA

● J. Daemen i V. Rijmen, “The Design of RijndaeL: AES - The Advanced Encryption Standard (Information Security and Cryptography)”, Springer, Berlin, 2002. ● S. J. Nielson i C. K. Monson, “Practical Cryptography in Python: Learning Correct Cryptography by Example”, Apress, Nju Jork, 2019.

● n.a, “Precisely”, https://www.precisely.com/glossary/encryption-keys, Precisely, n.d., Web, (01.03.2025. 17:34)

● Tryhackme i Strategos, “tryhacme”, https://tryhackme.com/room/cryptographybasics, Tryhackme Ltd, 21.10.2024., Web, (01.03.2025. 16:57)

- n.a., “Oracle Help Center”, https://docs.oracle.com/cd/E19047-01/sunscreen151/806-5397/i996724/index.html , Oracle Corporation, n.d., Web, (01.03.2025. 16:43)
- Malcolm Higgins, “Nordvpn”, https://nordvpn.com/blog/hashing-vs-encryption/, Nordvpn, n.d., Web, (01.03.2025. 16:58)
- n.a, “nexus group”, https://www.nexusgroup.com/difference-between-symmetric-and-asymmetric-cryptography/, nexus group, 25.07.2024., Web, (02.03.2025. 20:32)
- Josh Schneider, “IBM”, https://www.ibm.com/think/topics/cryptography-history, international Business Machines, n.d., Web, (04.03.2025. 19:35)
- AppliedGo, “Youtube” https://www.youtube.com/watch?v=gP4PqVGudtg&t=107s, n.p., 27.11.2017., Web, (14.03.2025. 19:59)
- Spanning Tree, “Youtube” https://www.youtube.com/watch?v=C4ATDMIz5wc, n.p., 22.08.2024., Web (14.03.2025. 20:01)
- n.a., “Geeks for Geeks”, https://www.geeksforgeeks.org/advanced-encryption-standard-aes/, n.p., 03.05.2025., Web, (10.03.2025. 18:42)
- vii02 i Legrandin, “Github”, https://github.com/Legrandin/pycryptodome, n.p., 14.04.2024., Web, (16.03.2025. 23:19)

5. BILJEŠKE

Datum predaje maturskog rada: _____

Komentar mentora o radu:

_____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Ocjena maturskog rada: _____ ( )

Datum odbrane maturskog rada: _____

Pitanja na odbrani maturskog rada:

1. _____

2. _____

3. _____

Ocjena iz usmene odbrane rada: _____ ( )

Komisija u sastavu: Potpis članova komisije: 1. _____, predsjednik

_____ 2. _____, ispitivač

_____ 3. _____, stalni član

_____